

# HW\_05

Henry Traynor

Due: 5:00pm, 29 September

Create a self-contained R script or Rmarkdown file to do the following:  
Simulate independent double observer data with the following properties:

- $n_{\text{Sites}}=200$
- $\lambda=3$
- $p_1=0.3$  and  $p_2=0.5$

```
set.seed(314159)
```

```
nSites <- 200
```

```
lambda <- 3
```

```
N <- rpois(nSites, lambda)
```

```
summary(N)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.00   2.00   3.00   2.91   4.00   8.00
```

```
mean(N)
```

```
## [1] 2.91
```

```
var(N)
```

```
## [1] 2.916482
```

```
p1 <- 0.3
```

```
p2 <- 0.5
```

```
pi1 <- p1*(1-p2)
```

```
pi2 <- (1-p1)*p2
```

```
pi3 <- p1*p2
```

```
pi4 <- (1-p1)*(1-p2)
```

```
cp <- c(pi1,
        pi2,
```

```

      pi3
    )

y <- matrix(NA,
            nrow=nSites,
            ncol=3
)

colnames(y) <- c("obs1_only",
                 "obs2_only",
                 "both"
)

for(i in 1:nSites) {
  counts <- rmultinom(1, size=N[i], prob = c(cp,pi4))
  y[i,] <- counts[1:3]
}

head(y)

```

```

##      obs1_only obs2_only both
## [1,]         0         1    0
## [2,]         2         0    0
## [3,]         1         1    0
## [4,]         0         1    0
## [5,]         0         4    0
## [6,]         0         0    0

```

```
summary(y)
```

```

##      obs1_only      obs2_only      both
## Min.   :0.000   Min.   :0.0   Min.   :0.000
## 1st Qu.:0.000   1st Qu.:0.0   1st Qu.:0.000
## Median :0.000   Median :1.0   Median :0.000
## Mean   :0.535   Mean   :0.9   Mean   :0.465
## 3rd Qu.:1.000   3rd Qu.:2.0   3rd Qu.:1.000
## Max.   :3.000   Max.   :4.0   Max.   :3.000

```

Fit the model using `unmarked`. You will need to create an observation covariate indicating observer A and B. When you create the `unmarkedFrameMPois`, use `type=double`.

```

observer <- matrix(
  c("A", "B"),
  nrow = nSites,
  ncol = 2,
  byrow = TRUE
)
colnames(observer) <- c(
  "observer1",
  "observer2"
)

```

```
library(unmarked)
umf <- unmarkedFrameMPois(
  y = y,
  obsCovs = list(observer = observer),
  type = "double"
)
```

## Warning: obsCovs contains characters. Converting them to factors.

```
fm <- multinomPois(
  ~ observer -1 ~ 1,
  data = umf
)

fm
```

```
##
## Call:
## multinomPois(formula = ~observer - 1 ~ 1, data = umf)
##
## Abundance (log-scale):
## Estimate SE z P(>|z|)
## 1.08 0.0741 14.5 8.36e-48
##
## Detection (logit-scale):
## Estimate SE z P(>|z|)
## observerA -0.66 0.128 -5.171 2.33e-07
## observerB -0.14 0.142 -0.989 3.23e-01
##
## AIC: 1254.228
## Number of sites: 200
```

```
coef_estimates <- data.frame(
  parameter = c("lambda_hat", "pA_hat", "pB_hat"),
  true_value = c(lambda, p1, p2)
)

coef_estimates$unmarked <- c(exp(coef(fm)["lambda(Int)"]),
  plogis(coef(fm)["p(observerA)"]),
  plogis(coef(fm)["p(observerB)"])
)

coef_estimates
```

```
## parameter true_value unmarked
## 1 lambda_hat 3.0 2.9354835
## 2 pA_hat 0.3 0.3406595
## 3 pB_hat 0.5 0.4650001
```

Fit the same model in JAGS

```

library(jagsUI)

model_string <- "
model {

  lambda ~ dunif(0,20)
  pA ~ dunif(0,1)
  pB ~ dunif(0,1)

  pi1 <- pA * (1 - pB)
  pi2 <- (1 - pA) * pB
  pi3 <- pA * pB

  for (i in 1:nSites) {
    y[i,1] ~ dpois(lambda * pi1)
    y[i,2] ~ dpois(lambda * pi2)
    y[i,3] ~ dpois(lambda * pi2)
  }
}
"

model_file <- tempfile(fileext = ".jags")

writeLines(
  model_string,
  con = model_file
)

jags_data <- list(
  nSites = nSites,
  y = y
)

jags_inits <- function() {
  list(
    lambda = runif(1),
    pA = runif(1),
    pB = runif(1),
    N=rowSums(y)+rpois(nrow(y), 2)
  )
}

jags_pars <- c("lambda", "pA", "pB")

jags_post <- jags.basic(
  data = jags_data,
  inits = jags_inits,
  parameters.to.save = jags_pars,
  model.file = model_file,
  n.chains = 3,
  n.adapt = 100,
  n.burnin = 2000,
  n.iter = 15000,

```

```
parallel = TRUE,
n.thin = 10
)

##
## Processing function input.....
##
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.255 minutes.
```

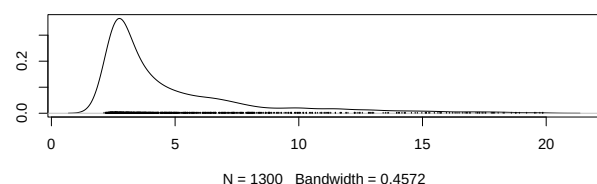
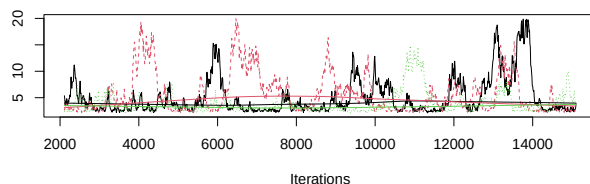
```
jags_summary <- summary(jags_post)

coef_estimates$JAGS_mean <- jags_summary$statistics[c("lambda", "pA", "pB"), "Mean"]
coef_estimates$JAGS_lower95 <- jags_summary$quantiles[c("lambda", "pA", "pB"), "2.5%"]
coef_estimates$JAGS_median <- jags_summary$quantiles[c("lambda", "pA", "pB"), "50%"]
coef_estimates$JAGS_upper95 <- jags_summary$quantiles[c("lambda", "pA", "pB"), "97.5%"]

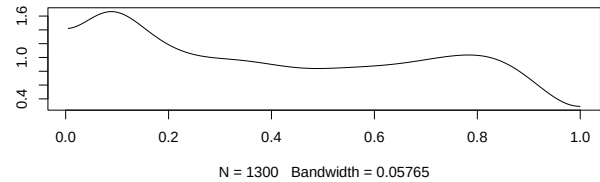
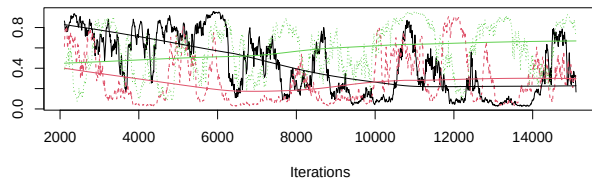
summary(jags_post[,jags_pars])
```

```
##
## Iterations = 2110:15100
## Thinning interval = 10
## Number of chains = 3
## Sample size per chain = 1300
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## lambda 4.8456 3.2806 0.052532      0.43928
## pA      0.4278 0.2843 0.004552      0.03921
## pB      0.4675 0.2890 0.004628      0.04145
##
## 2. Quantiles for each variable:
##
##           2.5%    25%    50%    75%    97.5%
## lambda 2.32103 2.6767 3.4726 5.6977 14.7410
## pA      0.03860 0.1560 0.3979 0.6913 0.9124
## pB      0.04957 0.1912 0.4536 0.7410 0.9310
```

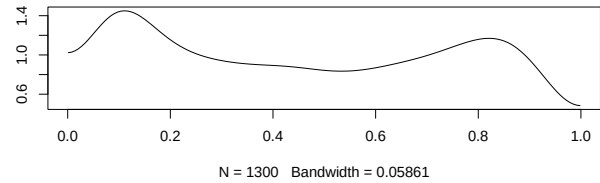
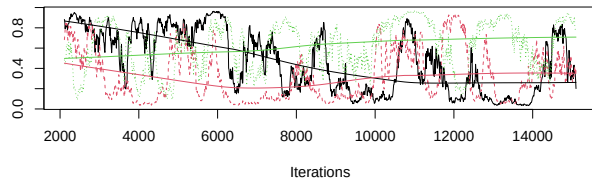
```
plot(jags_post[,jags_pars[1]])
```



```
plot(jags_post[,jags_pars[2]])
```



```
plot(jags_post[,jags_pars[3]])
```



```
library(coda)
```

```
##
## Attaching package: 'coda'

## The following object is masked from 'package:jagsUI':
##
##   traceplot
```

```
gelman.diag(jags_post)
```

```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## deviance      1.00      1.00
## lambda        1.07      1.18
## pA             1.59      2.45
## pB             1.58      2.44
##
## Multivariate psrf
##
## 1.41
```

```
data.frame(ESS = effectiveSize(jags_post))
```

```
##           ESS
## deviance 3900.00000
## lambda   51.48479
## pA       52.51886
## pB       47.53264
```

For both unmarked and JAGS, use the point estimates of  $p_1$  and  $p_2$  to compute the  $\pi$  probabilities.

```

pi_estimates <- data.frame(
  parameter = c("pi1", "pi2", "pi3", "pi4"),
  unmarked = c(coef_estimates$unmarked[2]*(1-coef_estimates$unmarked[3]),
    (1-coef_estimates$unmarked[2])*coef_estimates$unmarked[3],
    coef_estimates$unmarked[2]*coef_estimates$unmarked[3],
    (1-coef_estimates$unmarked[2]*(1-coef_estimates$unmarked[3])
  ),
  JAGS = c(coef_estimates$JAGS_mean[2]*(1-coef_estimates$JAGS_mean[3]),
    (1-coef_estimates$JAGS_mean[2])*coef_estimates$JAGS_mean[3],
    coef_estimates$JAGS_mean[2]*coef_estimates$JAGS_mean[3],
    (1-coef_estimates$JAGS_mean[2]*(1-coef_estimates$JAGS_mean[3])
  ),
  true = c(pi1, pi2, pi3, pi4)
)

pi_estimates$unmarked_delta <-
  pi_estimates$unmarked - pi_estimates$true

pi_estimates$JAGS_delta <-
  pi_estimates$JAGS - pi_estimates$true

pi_estimates

```

```

##   parameter  unmarked      JAGS true unmarked_delta  JAGS_delta
## 1      pi1 0.1822528 0.2278050 0.15    0.032252768 0.07780500
## 2      pi2 0.3065934 0.2675062 0.35   -0.043406571 -0.08249377
## 3      pi3 0.1584067 0.2000144 0.15    0.008406684 0.05001436
## 4      pi4 0.3527471 0.3046744 0.35    0.002747119 -0.04532559

```